



SARVA LABS • MOI FOUNDATIONS

The Linear Substrate and the Authority Layer

*Participant-Indexed Computation and the
MOI Authority Layer (MAL)*

Abstract

Information is Cartesian; value is linear. Double-spend is the copy map applied to value, and global consensus is the patch a Cartesian substrate needs to forbid it. Value, moreover, is not one thing: ordered by writer structure it forms four classes — owned, attested, contextual, and relational — and the Participant Layer is the substrate all four require. MOI represents value as a linear resource in a participant context — so the copy map, reentrancy, aliasing, and the need for global ordering do not exist in the category. The MDAG is the string diagram of that linear category, and every architectural property of the Participant Layer is a corollary.

Authority — the relational class, and the taxonomy's maximal case — is the existence proof. A grant of authority is a many-to-many relation with two independent writers — the one structure an ownership model cannot hold without surrendering parallelism, consistency, or revocation — and the agentic economy is the first regime to exercise it at machine speed by default. The second half of this paper develops the MOI Authority Layer (MAL): a stateful, revocable, auditable authority architecture for agentic systems, in which tokens are witnesses of valid state rather than bearers of permission. The remaining classes follow by containment.

Version 2.1 • August 2026

Anantha Krishnan • Founder, & Chief Architect, Sarva Labs

moi.technology

“Information can be processed anonymously. Value requires participants.”

Contents

Preface — From the Participant Layer to Its Foundations	3
Part I — Theory: Why the Substrate Is Forced	4
1. The conflation of computation and state	4
2. Information is Cartesian; value is linear	4
3. Consensus as a linearity patch	6
Part II — The Taxonomy of Value	7
4. The four classes of value	7
5. The ascent of writer structure: why authority is built first	8
Part III — The Substrate: The MDAG as a Linear Category	10
6. Participant as a dimension, not an owner field	10
7. The MDAG and its string-diagram reading	10
8. The Interaction Context Space	11
9. One context, inherited frames	12
10. Consistency and finality by construction	14
11. Where value lives: three state models	14
Part IV — The Authority Problem	17
12. Authority as a composed relation	17
13. The four failure modes of agentic authorization	18
14. Why ownership models face a trilemma	19
15. Revocation, delegation, and modulated trust	20
16. The orchestration demand	21
Part V — The MOI Authority Layer (MAL)	23
17. Overview and design commitments	23
18. What authority is not: three confusions	24
19. Architectural principles	26
20. The three-component composition	27

21. Vocabulary: mandates and their artifacts	28
22. Token taxonomy	28
23. The authority lifecycle	29
24. The offline discipline — a specified counit	31
25. Evidence and audit	32
26. MAL in action: one mandate, two worlds	33
Part VI — Against Today’s Systems	38
27. Four generations of agent authority	38
28. Properties compared	39
Part VII — Conclusion and the Series	41
29. Conclusion	41
The series	42

Preface — From the Participant Layer to Its Foundations

The lite paper, *The Participant Layer of the Internet*, makes some large claims and doesn't defend them. It says computation has been missing a dimension — the participant — and writes the fix as a function signature: $\psi(P, C, I) \rightarrow (P, C', V)$. It says mandates are enforced at the protocol level, that revocation is instant, that reentrancy isn't prevented so much as undefined, and that assets are conserved by construction rather than by audit. If you finished that paper wondering how any of this could actually be true, this is the paper where we try to show you.

The structure is simple. Parts I and II lay down one mathematical fact about the difference between information and value. Value turns out to come in four kinds, sorted by who can write to them. Part III derives the substrate that all four kinds need. Parts IV and V build the first real system on that substrate, the MOI Authority Layer, which handles the hardest of the four — authority itself. We start with authority partly because agentic systems need it right now, but mostly because it's the one case no ownership-based model can hold at all. If the substrate survives its hardest test, everything easier follows, and we'll try to make that containment argument precise enough to be checked.

A note on rigor. This paper sits between the lite paper and the formal CC series (CC0–CC6). We have tried to be exact in the claims and diagrammatic rather than proof-theoretic in the methods. Where a full proof exists, we point to it; where the argument here is informal, we say so.

Part I — Theory: Why the Substrate Is Forced

1. The conflation of computation and state

There's an assumption underneath modern computing that is so universal we rarely bother to name it: there is one computational space — a machine, a VM, a global state tree — and it exists before and independently of the people whose data it processes. "Who" is metadata. An address in a field, an owner column on an object, a row in an access-control table, a claim inside a token. The computation happens in a shared space, and identity is a label floating around inside it.

We want to be careful here, because the assumption isn't stupid. It's an engineering convenience with fifty years of success behind it, and much of our own professional history — years spent close to the core of exactly these systems — was spent profitably inside it. But we have come to think it is also a genuine category error, and that fixing it doesn't merely *suggest* a different architecture. It forces one.

The error is treating two different things as one thing: the state space (what is true) and the computational space (where transitions happen). They get fused because state is assumed to be anonymous. If state lives in a shared store, the machine that operates on it has to be shared too. But suppose state were indexed by participant instead. Then the computational space stops being global. For any given interaction, it becomes the union of the contexts of the participants actually interacting — and nothing more.

The rest of Part I is about why value, specifically, cannot live in an anonymous shared store without paying a price, and what that price is.

2. Information is Cartesian; value is linear

People usually distinguish information from value economically. You can make the distinction exactly, in category theory, and the exact version matters here, because everything that follows depends on it.

A symmetric monoidal category gives you two ways to combine processes: one after another (composition), and side by side, independently (the tensor product). Such a category is called *Cartesian* when every object comes with two natural, well-behaved maps: a copy map $\Delta : A \rightarrow A \otimes A$, and a delete map $! : A \rightarrow I$. Fox's theorem says this characterization is complete — Cartesian categories are exactly the symmetric monoidal categories where everything can be uniformly copied and thrown away.

Now read that in plain language. Something you can copy freely and discard freely is *information*. A file, a number, a fact: duplicate it and nothing is lost, drop it and nothing complains. The presence of the pair $(\Delta, !)$ is what it means, mathematically, to be information. And a category where those maps don't exist is a category of *linear resources*: things that flow along a wire, can't be forked, can't silently vanish. That is what value is like. So the two aren't neighboring concepts that happen to differ in practice. They are the two sides of one theorem.

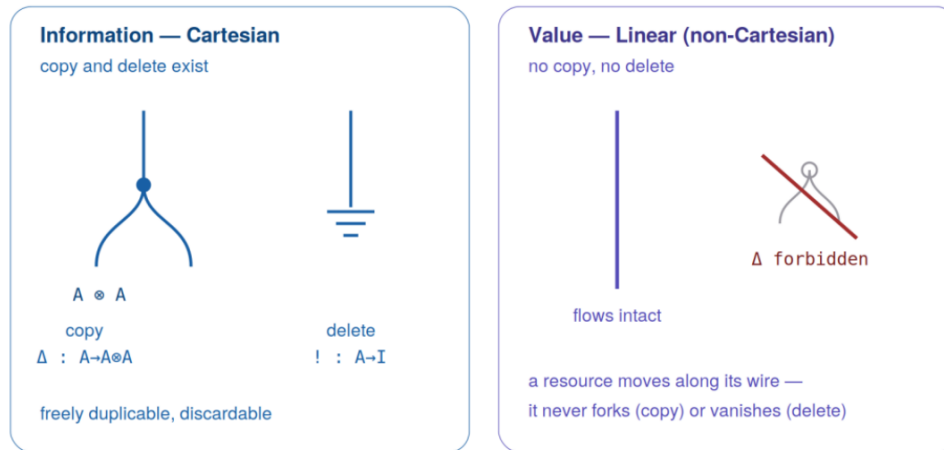


Figure 1. The two regimes as string diagrams. Left: information admits a copy node (Δ) and a delete/ground ($!$), so it duplicates and discards freely. Right: value is a linear resource — the copy map is absent, so a unit of value moves along its wire and never forks or vanishes.

Two other fields ran into the same boundary independently. In linear logic, contraction and weakening — use a hypothesis twice, discard one — are exactly copy and delete, and a linear type forbids both. In quantum mechanics, the absence of a uniform copy map is the no-cloning theorem. No-cloning of a quantum state and no-double-spend of a unit of value are, formally, the same statement: the category is not Cartesian.

For the careful reader: value lives in a symmetric monoidal category that is non-Cartesian, and if value is strictly conserved, also non-semicartesian — no natural delete either. That’s what separates it from a Markov category, which keeps a natural discard and models probability. A system may still permit *controlled* deletion — burning, expiry — as a modeling choice: a *specified counit on chosen objects, never a natural transformation on all of them*. We use this exception once, in Part V, for offline credentials. The degeneracy of collapsing value into a Cartesian setting is treated in CC2.

The thesis in one line

Information is Cartesian; value is linear. Every property that distinguishes a value-bearing substrate from an ordinary computer is a consequence of which side of Fox’s theorem its state lives on.

One definition has to be nailed down before the theorem gets used, because every classification in this paper depends on it. When we say something is value, we are never talking about its *content*. Content is information — bytes, numbers, text — and information copies; that’s the theorem. What’s linear is the *binding*: value is information bound to a participant in a context. An example. “The temperature in New York is 0°” is information. Copy it, email it, cache it — nothing is lost. “The temperature is 0° and I am in New York and I am cold” is value. Strip the participant out of that sentence and it doesn’t survive as a smaller fact; it collapses back into weather data. And running the collapse in reverse is precisely why a Cartesian substrate can’t hold value: it can store every byte of the content and still miss the binding, because the binding is exactly the part that doesn’t copy.

The binding test

Value is information bound to participant and context. The content copies; the binding doesn't. Detached from its participant, value degenerates into information about value. Every classification in Part II applies this test to the binding, never to the content.

3. Consensus as a linearity patch

With that distinction in hand, the central pathology of conventional blockchains becomes easy to state. A double-spend is not a security bug sitting on top of an otherwise sound design. A double-spend is the copy map Δ applied to value. The same unit spent twice is the same resource forked onto two wires — the one operation linearity forbids.

Now look at where the prevailing designs put value. On an account chain, it's a number in a globally addressable mapping. On an object chain, it's a field in an addressable object. Either way, a linear resource is being represented inside a Cartesian substrate — a store where copying is natively possible, because the store was built for information. The substrate permits the very operation that must never happen. So something has to forbid it after the fact.

That something is global consensus. By totally ordering every spend, the system guarantees the same unit isn't forked; the second fork is rejected as a conflict against the first. Consensus, in these systems, isn't fundamentally about agreement or fault tolerance for their own sake. It's a patch — the mechanism by which a Cartesian substrate is made to *simulate* linearity. And the cost of the patch, a global ordering that every operation must pass through, is paid by every transaction, related or not.

Today's object-centric chains eliminate some of this global ordering, through linear language types (linearity in the type system, not in the computational machine) and commutative state operations. But the escape extends exactly as far as one-writer state. The moment state has more than one writer — any shared object, any two-party structure — the object model falls back to global consensus, and the patch returns. So the object model doesn't refute the claim; it confirms it at better resolution: the consensus a system needs is proportional to how much of its state it cannot express as single-writer. Section 11 examines this in detail, and Part IV is about the class of state where writers necessarily multiply.

The reframing

Global consensus is the price of storing a linear thing in a Cartesian place. Make the substrate linear — let value be a resource that has no copy map to begin with — and there is nothing to globally order against. This is why we keep saying “less consensus, not faster consensus”: not a cheaper patch, but the removal of the condition that made the patch necessary.

Part II — The Taxonomy of Value

4. The four classes of value

Fox's theorem draws a boundary; this Part is about what lives on the linear side of it. If value is whatever fails the copy test *at the binding*, then value is not one thing — money — but a family of structures, and the family is a lot bigger than economics. We count four classes, presented in ascending order of how much they demand of the substrate. The measure of demand is *writer structure*: how many parties can change the state, and whether they do so independently of one another.

Owned value — one writer, one clock

Money, tokens, native assets. One participant, one writer: a linear quantity on a spine, mutated only by its holder's interactions. The digits of a balance are information and copy freely; the *holding* doesn't. This is the simplest class, and honestly it's the one the industry has spent fifteen years approximating — the account model simulates its linearity with global ordering, the object model with owner fields. It's also where ownership substrates come closest to adequate, because owned value is the one case where "one writer per cell" is actually true.

Attested value — two parties, one writer per artifact

Credentials, certificates, degrees, licenses. Two parties stand in the structure — an issuer who attests, a holder who holds — but each artifact still has a single writer: the issuer signs once and the artifact is fixed. What's linear here is not the document. A copy of a diploma is pixel-perfect. What's linear is the *attestation relation*: this issuer stands behind this claim about this participant, now. And the relation is live — degrees get revoked, licenses lapse, certifications get superseded. This is where bearer architectures start to crack: a copied credential is indistinguishable from a valid one precisely because the copy carries the content and drops the relation. Verifying against the relation rather than the artifact is, we would argue, the entire engineering problem of this class.

Contextual value — one participant, continuously written

Participant memory, preferences, trust history, accumulated agent context. There's no discrete artifact at all here — just a continuously evolving state made out of the participant's own history of interactions. You can copy a transcript of someone's memory; it's bytes. But the transcript is information *about* a participant, not the participant's live context. It's stale on arrival, its subject can't write to it, and it isn't accountable to the interactions that come after it. Contextual value raises the demand on the substrate because it never holds still: every interaction writes it, so it has to live where interactions land — on the spine itself.

Relational value — two writers, independent clocks

Authority. A many-to-many relation between participants — agent X may do Y on behalf of participant P — with a composite key and no single owner. Both sides change it independently: the

agent revises what authority *can* exist; each participant revises what authority *does* exist for them. Every earlier class had a home an ownership model could at least gesture at — a cell, an artifact, an owner. This one has none. Materialize the relation as an object and some third party ends up owning the intersection; shard it out and one of its two writers loses the ability to write. We come back to this failure in Part IV.

In every class, notice, the same thing is true: the content copies and the binding doesn't. What ascends from class to class is how much structure the binding carries.

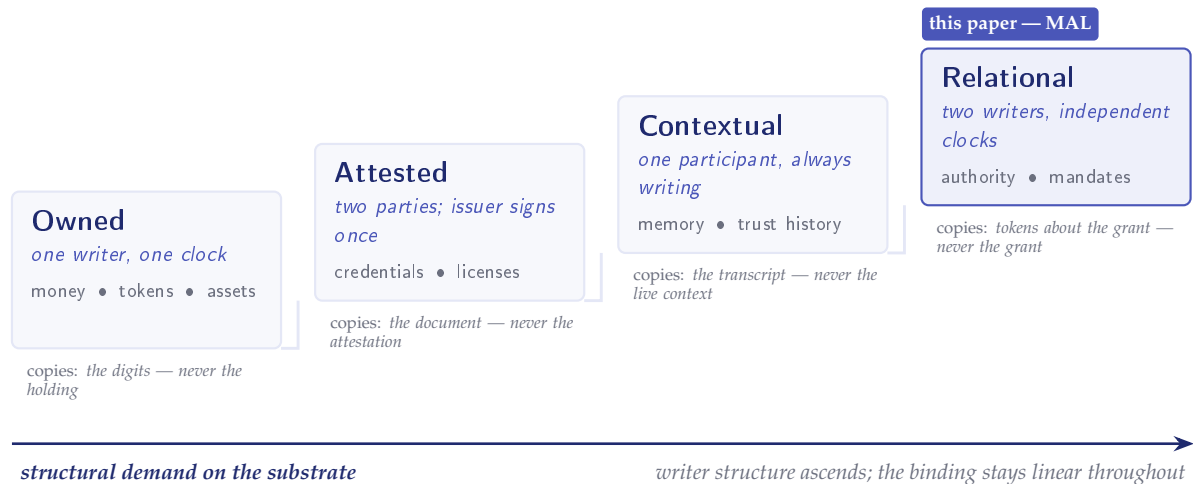


Figure 2. The taxonomy of value as a ladder of writer structure. Four classes in ascending structural demand: owned (one writer), attested (two parties, one writer per artifact), contextual (continuously written by its participant), relational (two independent writers on independent clocks). In every class the content is copyable and the binding is not; what ascends is how much structure the binding carries.

Class	Instances	Writer structure	Copyable (content)	Linear (binding)
Owned	money, tokens, native assets	one writer, one clock	the digits	the holding
Attested	credentials, licenses, claims	issuer signs once; holder holds	the document	the attestation relation
Contextual	memory, preferences, trust history	written by every interaction	the transcript	the live context
Relational	authority, mandates, delegation	two writers, independent clocks	tokens about the grant	the grant itself

Readers of the lite paper will recognize this list. The Context Superstate is exactly these four classes, held together under a single participant's address; Section 6 makes the identification exact.

5. The ascent of writer structure: why authority is built first

The ordering of the classes is structural demand, not importance. Each asks strictly more of the substrate than the one below it. Owned value needs a linear cell. Attested value needs a linear cell *plus* a live relation a verifier can reach. Contextual value needs the cell to be written by every interaction its participant has. Relational value needs all of that, twice, under two writers that never coordinate.

The classes are also cumulative in the other direction, and this is the argument that organizes the rest of the paper: a substrate that holds the relational class intact holds everything below it. Freeze one writer of the relation after a single signature and you get attested value. Collapse the relation to a single participant written continuously and you get contextual value. Reduce it to a quantity under one writer and you get owned value. We are stating these containments structurally here rather than proving them — the precise versions live in the CC series — but the *direction* is what matters for this paper. Prove the substrate at its hardest point and the rest comes along.

There's a second, entirely separate reason to start with authority, which is timing. The agentic economy is the first computing regime that exercises relational value at machine speed by default, because every agentic workflow is a delegation and every delegation is the two-writer relation. Section 16 makes that case in full. So the structural argument says authority is the hardest case, and the market happens to say it's the urgent one. It is fair to note that this convergence is convenient for the paper's thesis. We don't think it's really a coincidence, though — the reason agents strain today's authorization stack is precisely that they exercise the one class of value the stack was never shaped to hold.

Why authority comes first

Any of the Participant Layer's implementations could have been presented first. We present authority because it's the one no Cartesian substrate can hold at all, which means the implementation doubles as the existence proof. The substrate proved at the top of the ladder is proved for the whole ladder.

The remaining classes aren't deferred; they're pre-slotted. Each gets developed at its own depth in its own paper, as an instantiation of this taxonomy on the same substrate: participant-indexed exchange and native assets for the owned class, verifiable credentials for the attested class (the Proofly series), participant memory and agent context for the contextual class. This paper takes the relational class: the authority problem, and the MOI Authority Layer built to hold it.

Part III — The Substrate: The MDAG as a Linear Category

6. Participant as a dimension, not an owner field

If value must live in a linear substrate, the question is what gives the substrate its linearity. The answer is the indexing of state. In a conventional system the primary axis is the object or the account, and the participant is a field attached to it — findable by scanning and filtering, but not fundamental. The data is organized by *which object*; who is a lookup.

MOI inverts this. The participant is an axis along which the entire state space is partitioned. “What does this participant control” is not a filter over a global store but a coordinate: go to that participant’s context and the state is there, because state was laid out by participant in the first place. This is the concrete meaning of treating WHO as a first-class dimension rather than a field — not that participant identity is stored carefully, but that it is one of the coordinates by which all state is addressed. A field holds one value and admits one writer. A dimension is an axis the whole space is organized along.

In the vocabulary of the lite paper, the realization of a participant’s coordinate is the *Context Superstate*: the cryptographic structure holding the participant’s assets, trust relationships, storage, logic, keys, preferences, and intelligence as one verifiable, portable state. Throughout this paper, “the participant’s context” and “the Context Superstate” name the same thing at different levels of formality.

The Context Superstate is where the four classes of Part II actually live. The Assets sub-context holds owned value. The Trust sub-context holds relational value — Part V lives entirely inside it. Prefs and Intel hold contextual value, written by every interaction and resident where interactions land. Attested value anchors to the spine as verifiable claims whose relations stay live because the issuer’s context stays reachable. The lite paper drew this as a product diagram — seven sub-contexts. It is really the taxonomy: each class of value sitting where its writer structure demands.

7. The MDAG and its string-diagram reading

The data structure that realizes participant-indexing is the MDAG — the Multi-link Composite Directed Acyclic Graph. Each participant has their own directed acyclic graph: a vertical spine of points, one per state transition of that participant, ordered in that participant’s local time. These vertical edges are transitive links — a participant’s own causal sequence. When two participants interact, a new point is appended to each of their spines, and a horizontal interaction link is drawn between those two points. The global structure is a set of independent vertical spines, stitched together precisely and only where participants met.

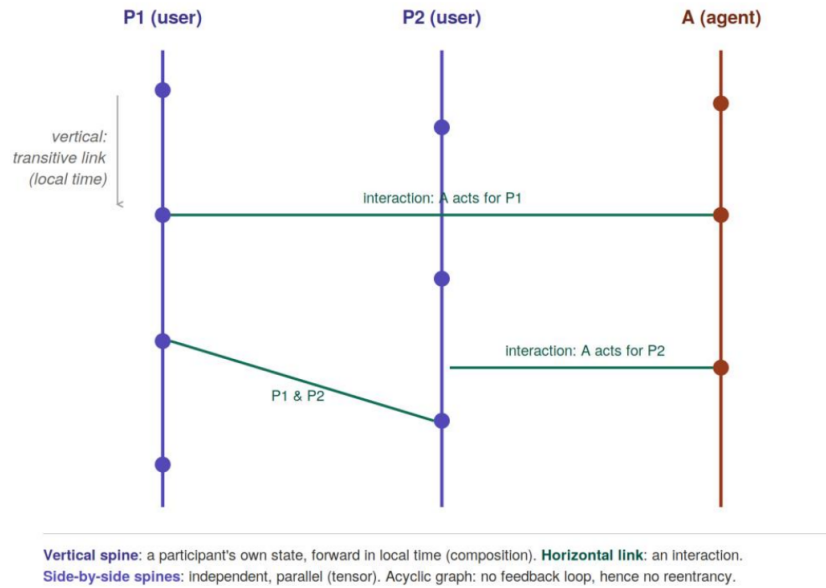


Figure 3. The MDAG. Vertical spines are participants' own histories; horizontal links are interactions. Two users and one agent: the agent acts for P1 and for P2 (two interaction links), and P1 and P2 interact directly. Spines that share no link are independent.

This structure is the string diagram of a symmetric monoidal category, and the correspondence is exact — this is not an analogy. A participant's context is an object — a wire. The vertical spine is sequential composition ($\circ$): the participant's state evolving forward. Placing independent spines side by side is the tensor product ($\otimes$): this is the parallelism, and it requires no coordination because tensored wires are by definition independent. An interaction link is a morphism that consumes the relevant wires and produces their successors, with the symmetry (the wire crossing) available when parties swap roles. And the acyclicity of the graph is the diagrammatic counterpart of the absence of a trace — no feedback loop — which is to say: no reentrancy. The properties one would otherwise have to engineer are read directly off the geometry the category already commits to.

8. The Interaction Context Space

Because state is participant-indexed, computation is no longer a global machine acting on shared data. The unit of computation is the union of the contexts of the participants in a given interaction — the Interaction Context Space — and it exists only for that interaction, only over those contexts. Computation is not a place that participants enter; it is what comes into being when participants meet.

This dissolves a category of contention that conventional systems treat as fundamental. When Jack transacts with John, and Alice transacts with Bob, the two interactions occupy disjoint Interaction Context Spaces; they share nothing to order, so they proceed in parallel as a matter of structure, not optimization. A global-state system serializes these two unrelated interactions anyway, because it has one state space and one order — contention it manufactures rather than discovers. The claim is deliberately narrow: MOI never manufactures contention. Interactions over disjoint contexts are

always parallel. The only serialization that can occur is between interactions that genuinely operate in the same context — real shared-resource contention, which any correct system must serialize, and which here localizes to the single shared context rather than escalating to the network.

No manufactured contention

Contention in MOI is exactly the real contention of the application, never an artifact of the protocol. A market with a single shared pool serializes on that pool — correctly, because everyone truly touches it. A market expressed as disjoint bilateral interactions is embarrassingly parallel. Exploiting the available parallelism is an application responsibility; the protocol's commitment is to add none of its own.

9. One context, inherited frames

The Starbucks model. The prevailing assumption — that entering a shared space means sharing state — is best seen by stating it physically. Walking into a café is entering a shared context: there is a menu, a counter, a queue. It does not mean the patrons throw their wallets into a communal jar and then queue to take their own money back out. Yet that communal jar is precisely what a global-state system does: the moment a participant interacts in a shared venue, their balance is treated as part of the shared mutable state that every other participant's transaction must be ordered against. Put physically, it is absurd — no one would agree to surrender their wallet on walking through the door.

The correct decomposition is the one any café already uses. State inside the shared context sorts into three kinds, and they stay distinct. The venue's own state — the menu, the prices — belongs to the venue's context (the venue is itself a participant, whose context the patron inherits on entering). Each participant's own state — their wallet, their loyalty points, their history — remains theirs, participant-indexed, untouched by mere presence in the room. And the one genuinely shared, contended resource — the single cashier — is the only thing that requires a queue, because two people truly cannot consume the same teller-moment at once. The line forms at the register and nowhere else: a patron waits for the cashier, never to reach their own pocket.

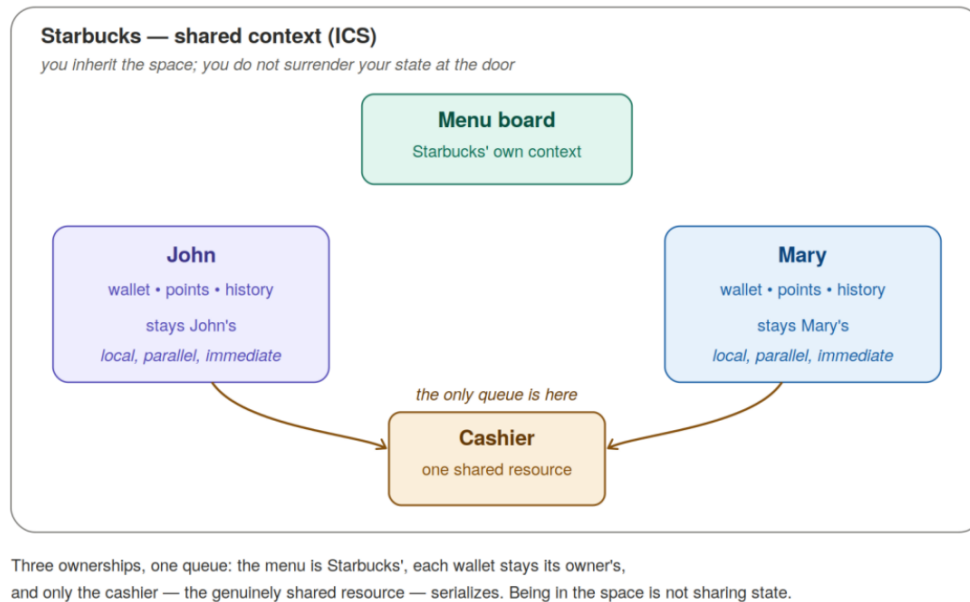


Figure 4. The Starbucks model. A shared context contains three ownerships: the venue's state, each participant's own state, and the single contended resource. Only the contended resource serializes. Being in a shared space is not the same as sharing state — the space is shared; the state is not.

This is the whole of the Interaction Context Space in one image, and it locates contention correctly: contention attaches to a resource genuinely consumed by more than one party, never to co-presence in a context. A global-state system serializes everything in the venue because its substrate cannot tell the cashier apart from a patron's wallet — lacking a participant dimension, the only relation it can express is “in the same space,” which collapses to “shares all state.” Participant-indexing distinguishes them structurally: the wallet travels with its owner because the owner is a dimension, not a row in the room's table. One venue, three ownerships, one queue — and the queue is only ever as long as the real contention, never as long as the room.

A participant is not fragmented into independent sub-identities. Each participant has exactly one context. A role is an inherited context: when Jack acts as a rider, he inherits the rider context of that relationship; when he watches a game, he inherits the venue's context. He is one participant operating within different inherited frames — the way a person behaves within the frame of a café and, simultaneously, within the frame of a match on their phone. The frames are separate worlds, so acting in both at once creates no conflict.

This answers the apparent hard case — two interactions that both “touch Jack.” They do not contend, because they operate in different inherited contexts, which hold different state and even different assets: a balance in one role's context is not the same cell as a balance in another's. Value moves between a participant's contexts only by an explicit transfer, which is itself an ordered, witnessed interaction — a movement of the resource along its wire, never an ambient read of a shared pool, and never a duplication. The unit of mutual exclusion is therefore the context actually written, not the participant; two interactions conflict only when their written contexts genuinely intersect.

The load-bearing invariant

Inheriting a context grants participation in that frame, never a writable link back to the participant's other contexts. All cross-context effects are explicit transfers. The primary context is never mutated as a side effect of acting in an inherited one. This single invariant — no implicit reach-back — is what makes the linearity hold operationally: it is the absence of the copy map, enforced as a rule about what an interaction may write.

10. Consistency and finality by construction

With no implicit reach-back, the classic bug-classes of concurrent shared-state systems simply don't arise. Reentrancy requires a copyable, globally-addressable handle through which execution re-enters a context mid-update; in a linear structure no such duplicable handle exists, so there is nothing to re-enter through — reentrancy is unrepresentable, in the same way cloning is unrepresentable for a quantum state. Inconsistency from aliasing requires two copies of one reference observing each other's partial writes; aliasing is the copy map on references, and with no copy map there is nothing to alias. Consistency here isn't a discipline anyone follows. The operations that would break it don't exist.

Finality is the same locality read in time. Because an authority or value decision resolves entirely within the contexts it concerns, it is final the moment those contexts agree — no global view ratifies it, no block buries it, no quorum weighs in on a fact that involves only the parties to it. A revocation is final to its issuer immediately, because no third party's agreement was ever part of what made it true. Hyper-local finality and participant-scoped parallelism are not two features; they are the spatial and temporal readings of one fact — that the truth lives in the participant contexts and nowhere else.

At the network level, this locality is what KRAMA implements: for each interaction, the witness nodes associated with the participants form an ephemeral validation cluster, certify the outcome, and dissolve. The protocol does not replicate and reconcile; it *witnesses and certifies*. Every authority decision in Part V's MAL is one of these witnessed, locally-final interactions.

11. Where value lives: three state models

The taxonomy in this section is of state models, not products. Where value lives — in a shared table, in discrete owned objects, or on the participant dimension — is a representational choice that precedes any blockchain, and it spans databases as readily as chains: a bank's core ledger is the account model, and a document store such as MongoDB holds the owned-object pattern with the owner as a field. Better engineering doesn't escape the choice either. Spanner is the account model perfected — externally consistent, globally serializable — and value in Spanner is still rows in a Cartesian store: conservation is a constraint the application checks, not a structure the substrate has, and the ordering cost is paid in commit-wait rather than consensus rounds.

There is a second limit that no engineering reaches. Every database, however consistent, is a single

trust domain — the trust lives in the administrator, not the substrate. The moment value must move between mutually distrusting parties, the substrate ends at the boundary and transfer degrades to message-plus-reconciliation. Reconciliation is consensus at human speed: the patch of Section 3, paid in clerks and settlement windows instead of validator rounds. The three models below are therefore examined through their cleanest expressions — Ethereum, Sui, and MOI — as instances of design classes, not as a comparison of products.

The account model — value in a shared table

The account model holds value as a quantity in a globally shared table; Ethereum is its cleanest expression. On an account-based chain, a token is a single contract holding a mapping from address to balance. A holder's balance is an entry in that one contract's storage — a row in a shared table, keyed by address. Because every transfer writes the same contract's state, transfers are totally ordered against one another even when they share no economic relationship. The representation is clean — a balance is a single integer, so there is no fragmentation — but it is a linear resource held in a Cartesian, globally-shared store, and the price of preventing the copy map is exactly the global ordering of Section 3: a clean representation, paid for with global serialization.

The owned-object model — value in discrete objects

An object chain such as Sui partitions state by ownership. Each object has one owner, and a transaction touching only objects a party owns can take a consensus-less fast path, because no one else can touch those objects. Value is modeled as owned coin objects carrying a value field; a transfer is the splitting and merging of objects rather than the editing of a row. This is a genuine generalization of the UTXO model from coins to arbitrary typed objects, and it delivers real parallelism for independently-owned state.

It carries two structural costs. First, fragmentation: because value lives in discrete objects, repeated spending leaves a growing population of small remainder objects that must be periodically merged — a maintenance overhead the ledger model never incurs. This is the price of the owned-object representation, just as serialization is the price of the shared-table one; neither is free, and they are mirror images. Second, and more consequential for what follows: the owner is still a field on the object. The object is the first-class thing; the participant is an attribute that says who may move it. That works flawlessly for state with a natural single owner — a coin — and strains precisely when state has no single owner. This is the boundary Section 3 identified — the escape from global ordering ends exactly where single-writer state ends — and Part IV is about what lies past it.

The participant-indexed model — MOI

MOI sits where neither model can. Value is a quantity held on the participant dimension — a clean number, like the account model, so there is no fragmentation; but the context is participant-partitioned rather than a shared global table, so transfers are independent and locally final, like owned objects. The two costs each of the other models pays are both artifacts of storing value off the participant axis: in a shared object it serializes, in a split-able object it fragments. Put the quantity on the participant dimension — neither shared nor splittable — and both costs vanish at

once.

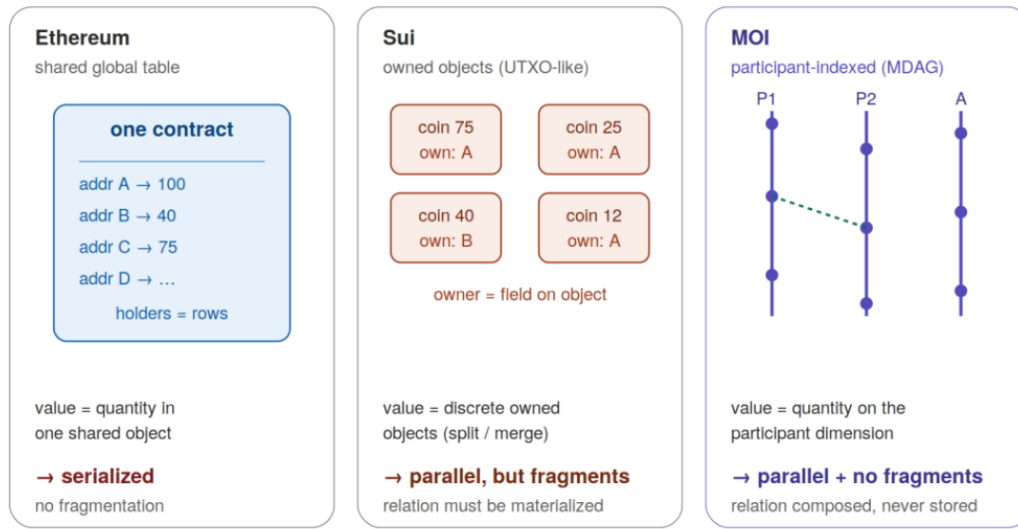


Figure 5. Three representations of the same token, by model. The account model (Ethereum): a quantity in one shared contract — no fragmentation, but serialized. The owned-object model (Sui): discrete owned objects — parallel, but fragmenting, with the owner as a field. The participant-indexed model (MOI): a quantity on each participant’s spine — parallel and unfragmented, with the participant as the indexing dimension and relations composed rather than stored.

Property	Account model	Owned-object model	Participant-indexed
Exemplar	Ethereum	Sui	MOI
Partitioned by	nothing (global)	object ownership	participant
Value representation	row in shared table	discrete owned objects	quantity on spine
Parallelism	none (serialized)	owned-object fast path	disjoint contexts
Fragmentation	none	yes (split / merge)	none
Ordering needed	global	per shared object	only real contention
Participant is	a field (address)	a field (owner)	a dimension
Many-to-many relation	junction state	materialized object	composed at action

Part IV — The Authority Problem

12. Authority as a composed relation

Part II placed authority at the top of the ladder. This Part shows what goes wrong when systems try to hold it anyway. Authority is the case the object model cannot absorb — which is why it serves as the existence proof for participant-indexing. Authority is not a thing one party owns; it is a many-to-many relation between an agent and a participant — agent X may do Y on behalf of participant P. It has a composite key and no single owner. And both sides mutate independently: the agent redefines what authority can exist (its schema), which should affect all participants at once; each participant grants, narrows, or revokes their own authorization, which should affect only them.

In MOI this relation is never stored. The agent's side — the schema, content-addressed by hash — lives on the agent's spine; the participant's side — the grant — lives on the participant's spine. Validity is computed as a join across the two contexts at the moment of action: an action's witness is valid if and only if the agent's current schema hash and the participant's current grant both still agree with what the witness was issued against. The relation is the horizontal interaction link of Section 7 — composed where the two spines meet, evaluated, and never materialized as an entity that some third party must own.

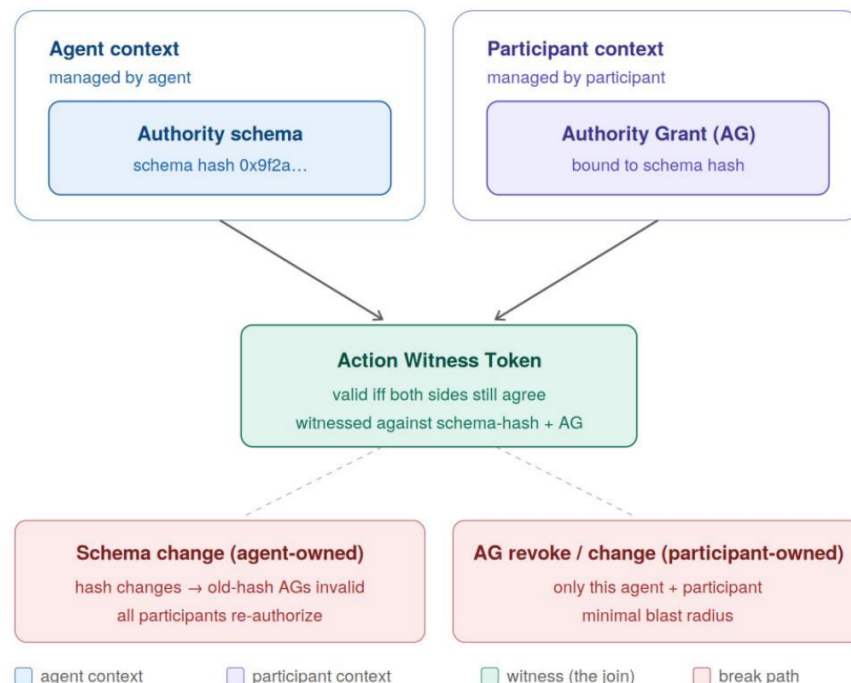


Figure 6. Authority as a two-context join. The agent's schema (content-hashed) and the participant's Authority Grant live on separate, independently-owned spines. A witness is valid only while both agree with what it was issued against. The two break paths are scoped by ownership: a schema change invalidates every old-hash grant at once; a grant change touches only that one agent-participant relationship.

The two break paths are scoped by ownership, and the scoping falls out correctly. A schema change is agent-owned: the hash changes, so every grant bound to the old hash stops resolving at once — all participants re-authorize, which is precisely the blast radius a schema change should have. A grant change is participant-owned: it touches exactly one agent-participant relationship and nothing else — the minimal blast radius. Neither party holds a lever that reaches into state the other owns.

One example runs through the next three sections — the lite paper’s own scenario. *Asha tells her travel agent: “Book me the cheapest Delhi–London flight, under \$800, pay with my Visa ending 6411, before Friday.”* That sentence is a mandate: scoped, bounded, time-limited, revocable. The same mandate fails under bearer authority, fails three different ways on an ownership substrate, and holds on MOI. Sections 13 through 15 walk through each.

13. The four failure modes of agentic authorization

Agentic systems are where these failures stop being academic. An autonomous agent acts on behalf of a principal across multiple systems, over time, with discretion. Four failure modes recur across every current approach.

Failure 1 — Authority drift. A token issued at time T_1 reflects the principal’s intent at T_1 . By time T_2 the principal may have changed their mind, but the token does not know. Bearer tokens, JWTs, and OAuth scopes all suffer this. The verifier honors the token because it cannot tell the difference between a still-valid grant and a stale one.

Failure 2 — Revocation latency. When the principal revokes, the revocation must propagate to every system that might honor the token. Revocation lists, short TTLs, and gossip protocols all narrow the window but do not close it. The system designer must choose: short TTLs (high friction) or long TTLs (long revocation windows). There is no clean exit.

Failure 3 — Accumulating capability. Bearer capability systems — UCAN, Biscuit used naively, capability tokens in object-capability systems — allow agents to accumulate authority over time and combine it in ways the original principal never explicitly approved. The resulting authority surface is computable but not human-auditable.

Failure 4 — Opaque sub-delegation. When an agent invokes a sub-agent, the principal often loses visibility. Either the sub-agent inherits ambient authority (bad), or it requires explicit per-invocation consent from the principal (impractical), or the system invents a delegation hierarchy that the principal cannot verify (worst). None of these compose with autonomous workflows.

Why the standard answers fall short

OAuth and JWT were designed for human-to-service authentication, not agent-to-agent action. RBAC and ABAC assume identities are stable and roles are slow-moving. Object-capability systems assume bearer authority is acceptable. Zero-trust architectures assume the verifier can always reach the policy decision point. Agentic systems violate every one of these assumptions. The authority layer for agents

has to be designed for the agentic case from first principles.

Worked example — Asha’s mandate under bearer authority

The agent receives an OAuth-style token scoped to “travel + payments,” \$800. **Drift:** on Tuesday Asha lowers her budget to \$500; the token still says \$800, and no verifier can tell a stale grant from a live one. **Latency:** on Wednesday she revokes entirely; the airline’s API honors the token until its TTL runs out on Friday — the booking she cancelled can still be paid for. **Accumulation:** over months the agent quietly holds tokens for flights, hotels, and refunds, and can combine them into authority Asha never reviewed as a whole. **Opaque sub-delegation:** the agent hands the payment step to a sub-processor Asha cannot see, bound, or audit. None of this is implementation sloppiness. All four failures are the same failure: the token carries the authority instead of witnessing it.

14. Why ownership models face a trilemma

This is the failure Part II predicted for the relational class. An ownership model has only one way to hold a composite-keyed, two-writer relation: materialize the intersection as a managed entity. That forced materialization is the root of every difficulty above, and it presents as a trilemma in which each available structure sacrifices one of the three things authority actually requires.

- Model the relation as one shared object, and the representation is correct — but every participant of that agent writes the same object, so they serialize against one another. Parallelism is lost precisely at the participant boundary.
- Sharding the relation into per-participant owned objects makes the participant side parallel. But now the agent cannot mutate its own side across objects it does not own, and a schema change cannot propagate without reintroducing a shared object. Agent-side consistency is lost.
- The third option is bearer capabilities the agent issues and participants hold — elegant until revocation, which a bearer token cannot honor without an indirection cell that is itself a shared object, or a hand-built membrane. Clean revocation is lost.

Parallelism, agent-side consistency, clean revocation: an ownership model yields any two. The reason is structural, not a missing feature — ownership assumes one writer per cell, and authority is a cell with two writers on independent clocks. One can approach MOI’s shape with an object per party and attempt to join them at runtime, but the join must discover, coordinate, and atomically snapshot the pair, and all three needs converge on a single shared registry that is consensus-bound and globally serializing. That registry is the materialized intersection coming back in through the side door. And it is not removable: composing a many-to-many relation without a shared coordination point requires the participant to be a dimension of the substrate — the one thing an ownership model cannot retrofit.

Worked example — building Asha’s mandate on an ownership substrate

Try to hold the same mandate on an object chain. **Attempt (a), one shared mandate object for the agent:** every customer’s grant writes the same object, so Asha’s Tuesday budget change queues behind thousands of strangers’ bookings — parallelism lost. **Attempt (b), one owned mandate object per customer:** Asha’s side is now parallel, but when the agency upgrades its schema — adds rail bookings, changes its payment flow — it cannot reach into objects it does not own; the change cannot propagate without reintroducing a shared registry — agent-side consistency lost. **Attempt (c), a bearer capability Asha signs and the agent holds:** elegant until Wednesday’s cancellation, which the capability cannot honor without an indirection cell that is itself a shared object — clean revocation lost. The trilemma is not bad engineering. It is what a one-writer-per-cell substrate does to a two-writer relation.

15. Revocation, delegation, and modulated trust

Because the relation is composed from two owned contexts rather than stored, its operations inherit the linearity of the substrate. Revocation is a withdrawal in the participant’s own context: the join simply stops resolving, instantly and locally, with no bearer token to chase and no list to distribute — and it propagates down any delegation chain by the same mechanism, because a sub-grant is an attenuated link that depends on the grant above it. Delegation is bounded and structural: an agent hands off a narrower slice of authority, and the receiving party’s permission is verified against the original grant, not against the first agent’s assertion. Scoping here is structural rather than policy: no party holds a control that reaches beyond the context it owns, so over-broad revocation isn’t something the system carefully avoids — it can’t be expressed at all.

One further property matters enormously for regulated domains. Because the validating context is anchored to the participant, the trust and compliance regime is a property of the substrate, not configuration sitting on top of it. A participant in one jurisdiction is validated under that jurisdiction’s regime; a participant in another, under theirs — while the same agent serves both. A monolithic chain has one global validator regime, so jurisdictional compliance can only ever be application data on a non-compliant substrate. Participant-anchored validation makes the compliance regime per-participant at the infrastructure level: one agent, several participants, several regimes, each enforced where it is validated. Context does not only provide parallelism; it provides modulated trust.

Worked example — Asha’s mandate on the participant-indexed substrate

On MOI the mandate is a grant on Asha’s spine; the agent’s schema hash is on its own. Tuesday’s budget change is one state advance in her context — every action from that moment witnesses against \$500, with nothing to propagate. Wednesday’s revocation is final the instant she issues it: the join of Section 12 simply stops resolving; there is no token to chase, because the token was never the authority. The payment subagent acts under a delegated token provably inside her \$800/Visa-6411 bound — narrower, never broader — and every hop

appears in her evidence log. And when the same agent serves Asha in Delhi and Ben in London, her interactions validate under India's DPDP regime and his under the UK's — one agent, two participants, two regimes, each enforced where it is validated. Parallel, consistent, revocable: all three, because the relation is composed, not owned.

16. The orchestration demand

Sections 12 through 15 are timeless: the two-writer relation was exactly as impossible to hold on an ownership substrate a decade ago, and nobody built an authority layer for it then. What changed is the load. Agents began spawning agents, and a decade-old structural gap became acute.

The topology shift

The authorization stack in production today was built for a static topology: one human, one application, one session. Its unit of consent is the login; its unit of authority is the scope; its clock is the human's. Agentic systems have a different shape. An orchestrator receives a goal, decomposes it at runtime, and spawns subagents that did not exist when consent was given — each needing not the principal's authority but a *slice* of it, narrower than the parent's, for minutes rather than months, often across an organizational boundary where the verifier has never seen the principal. The many-to-many relation of Section 12 stops being an edge case and becomes the default topology of every workflow: one principal, one orchestrator, N transient subagents, M counterparties — with N chosen by the plan, not by the person.

How the stack copes today

Three patterns are observable in production agent systems, and each is a failure mode of Section 13 industrialized. First, **key inheritance**: the orchestrator's bearer credential is copied down the tree, because a subagent that did not exist at consent time has nothing else to present. Drift and accumulation (failures 1 and 3) become transitive — every spawn widens the set of holders of the principal's full authority, and the principal's mental model of "who can act for me" is wrong within minutes of the first fan-out. Second, **transport without authority**: the emerging agent-to-agent protocol layer — messaging, discovery, capability description — is solving how agents find and address one another while explicitly deferring what an agent *may do* to "the application layer." The deferral is honest — and it leaves the middle of the stack empty: the transport standards are converging while the authority they transport remains a copied, opaque blob. Third, **registry identity**: agent registries, verifiable agent identifiers, and know-your-agent schemes give the fan-out tree names — recognition at scale — which sharpens accountability and changes nothing about control. Part V returns to both patterns.

The arithmetic of the window

Every bearer architecture has a revocation window — the interval between the principal's change of mind and the last verifier's knowledge of it. At human speed the window was tolerable: a session's worth of actions fit inside it. The window's cost, however, is not measured in time; it

is measured in *actions*, and the number of actions inside a window scales with agent throughput, not with the window's length. Hold the TTL fixed and exposure grows linearly with actions per second. Shorten the TTL and re-issuance traffic grows instead, re-creating the coordination cost the tokens existed to avoid, at every node of the fan-out tree. There is no setting of the dial at which a bearer window is safe at machine speed, because agents manufacture actions and the window prices in actions.

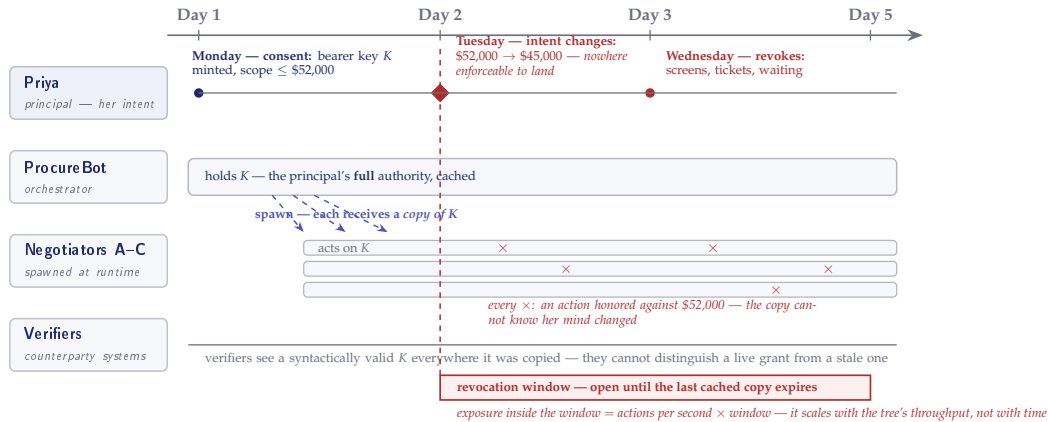


Figure 7. The orchestration topology under bearer authority. The orchestrator's credential is copied to runtime-spawned subagents; the Tuesday change of intent has nowhere enforceable to land; the Wednesday revocation opens a window that closes only when the last cached copy expires. Figure 11 replays this topology on MAL.

The demand

The agentic economy is therefore the first computing regime in which relational value — the maximal class of Part II — is exercised at machine speed by default. Owned value moved to machine speed first (payments); attested value is moving (credentials); but agents force the relational class, because every agentic workflow is a delegation, every delegation is the two-writer relation, and every Cartesian mitigation is one of the trilemma arms of Section 14. Section 5 argued authority is the structurally hardest class; this section has argued it is the commercially urgent one. A substrate proving itself has no better place to start.

Part V — The MOI Authority Layer (MAL)

From theory to system

The two-context join of Section 12 is not hypothetical. This Part specifies the system built on it: the MOI Authority Layer, in which the join is evaluated by a decidable policy engine, witnessed by the network, and surrounded by an evidence discipline that makes every authority decision reconstructable. Each design decision in Part V traces back to something Parts I through IV established.

17. Overview and design commitments

The MOI Authority Layer is the trust enforcement architecture for agentic systems built on the Participant Layer. It answers four operational questions about every agent action: what is the agent permitted to do, who granted that permission, is the action within scope at this moment, and can it be revoked. Existing approaches answer one or two of these. MAL answers all four by design, and produces cryptographic evidence for each answer.

The core claim

MAL treats authority as a stateful relationship between a participant and an agent, not as an information artifact embedded in a token. This single inversion — the operational form of “value is linear, information is Cartesian” — eliminates an entire class of authorization failures: stale credentials, undetectable revocation, accumulating bearer authority. It also reframes audit as a complete, reconstructable record rather than a partial log.

Four design commitments govern the layer. Each is the operational restatement of a structural fact from Parts I through IV.

1. **All authority is stateful.** Authority lives in the participant’s context on MOI — in the Trust sub-context of the Context Superstate — never in tokens. Tokens are witnesses of valid state, not bearers of permission. (This is Section 12’s composed relation, enforced.)
2. **Authority is always revocable.** Participants can revoke any granted authority at any time. Revocation propagates instantly to all dependent capabilities, including sub-delegated ones. (This is hyper-local finality, Section 10, applied to the grant.)
3. **Offline action requires explicit, scoped, time-bound participant consent.** The system does not silently issue offline capability. Participants are told what they are accepting and what risk window they are accepting it for. (This is the specified cunit Part I licensed — the one exception to linearity, surfaced rather than hidden.)
4. **Every action is reconstructable from evidence.** The audit log answers not just what happened, but why it was permitted at the moment it happened — including for actions taken during participant-accepted offline windows. (This is the MDAG read as a record: the witness structure is the evidence structure.)

18. What authority is not: three confusions

The failure modes of Section 13 and the trilemma of Section 14 share a root that is conceptual before it is technical: three distinct things are routinely collapsed into the single word “authority,” and every current approach is engineered on top of the collapse. MAL is the system that results from pulling the three apart.

The collapse has a definitional form. In prevailing systems, authority is a pre-existing policy: static, organization-authored, one writer, applied to subjects described by attributes. In MAL, authority is a live relation: dynamic, stateful, per-interaction, between two participants — a registered agent and the participant it acts for. The prevailing response to agentic speed has been to make the decision dynamic — runtime evaluation, fresh attributes — because the authority itself could not be. MAL makes the authority dynamic, which is why the decision can be deterministic: a mechanical witness of the relation as it stands, rather than an intelligent judgment about an artifact authored elsewhere.

Confusion 1 — Information about authority mistaken for authority

A bearer token, a JWT, an OAuth scope, a capability — each is information about a grant: a snapshot, taken at issuance, of what the principal intended at that moment. Information, by Part I, is Cartesian: it copies freely, caches freely, presents freely — and therefore goes stale freely, because nothing binds the copy to the relation it describes. Authority itself behaves like value: it is a stateful relation that exists in exactly one place (the participant’s context), changes the instant its owner changes it, and cannot be duplicated without becoming counterfeit. Every system that puts the authority in the token has stored a linear thing in a Cartesian artifact — the category error of Section 3, replayed at the application layer. Drift, revocation latency, and accumulation are not three separate bugs; they are the copy map applied to authority three different ways.

Externalizing the artifact does not escape the confusion; it relocates it. Moving authority from the token to a policy decision point makes the decision fresh while leaving the authority what it was — a one-writer artifact, authored at administrative cadence, in which the participant appears as data rather than as a writer. The relocation has now happened twice: from the wire (the bearer token) to the server (the policy store). Each move fixed the previous address’s staleness and inherited the category error at the new one — and the server address adds a failure the wire did not have: every enforcement point must be able to reach the decision point, the assumption Section 13 already names. A stateful decision about a static authority is not stateful authority. The loop closes only when the stateful thing is the relation itself — two writers, indexed by the pair, advancing at interaction cadence — at which point evaluation needs no cleverness, because every interaction feeds back into the relation it was authorized against.

The first formulation

Information about authority is not authority. The token is information about the grant — Cartesian, copyable, stale-able. The grant is the authority — linear, stateful, owned. This is the thesis of Part I specialized to the relation that governs agents: a token can witness the relation; it cannot be the relation.

Moving the artifact from the token to the policy server changes its address, not its category.

Confusion 2 — Access control mistaken for authority: the speed mismatch

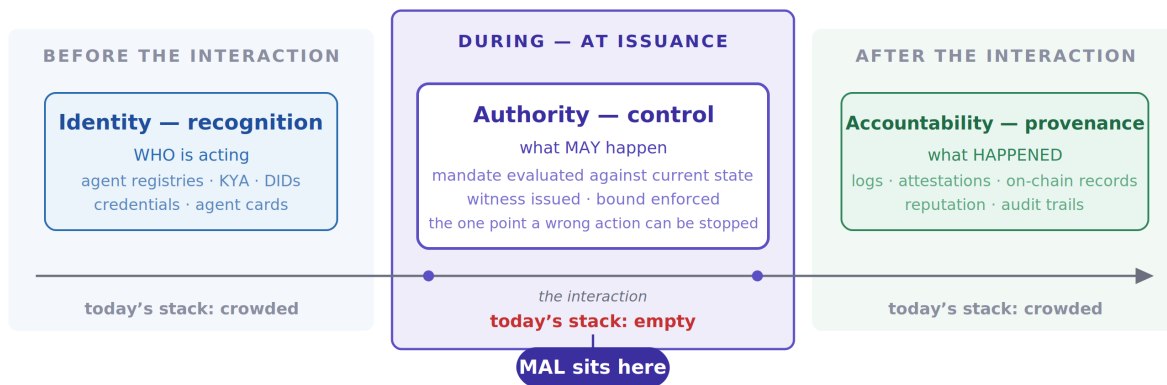
Agents moved logic to machine speed; authority stayed at human speed. The grain of today's access control is the human consent event — a screen, a click, a session — producing coarse scopes authored by a person and enforced by nothing finer. So when an autonomous agent acts a thousand times an hour, the system offers a false choice: keep the human in the loop per action, and forfeit the autonomy the agent was built for; or grant broad standing access once, and forfeit the safety. Both losses flow from confusing access control — session-grained permission to enter — with authority — interaction-grained control over what may happen.

MAL dissolves the dilemma by splitting what today is fused. Intent is **authored once, at human speed** — the mandate, with its bounds expressed in the participant's own language at consent time. Enforcement then runs **per interaction, at machine speed** — the surface evaluation and the witness issuance, with no human in the loop and none needed, because the human's judgment is already in the bound values being evaluated. The human is in the loop of authorship, never in the loop of enforcement. That split is what lets delegated authority run at the speed of the agents exercising it.

Confusion 3 — Identity and accountability mistaken for authority

The agent-infrastructure field is converging on the two ends of the interaction. Before it begins: identity — agent registries, know-your-agent schemes, decentralized identifiers — recognition of who is acting. After it ends: accountability — provenance, attestations, on-chain logs — reconstruction of what happened. Both are necessary; neither is control. Identity without authority is a well-recognized agent with unbounded power. Accountability without authority is a cryptographically perfect record of a harm that nothing prevented. Authority is the middle of the interaction: pre-interaction control over what may happen — the only one of the three that stops a wrong action before it occurs (Figure 8).

MAL occupies that middle, and nothing is traded for it: because every authority decision is a witnessed state transition (Section 25), accountability falls out as a corollary — the audit trail is the data structure, not a system bolted beside it. The field is building the bookends. Authority is the book.



Identity cannot stop a wrong action. Accountability records it after the fact. Only authority prevents it — and because every MAL decision is a witnessed state transition, accountability falls out as a corollary of control.

Figure 8. Where authority sits. Identity (recognition) crowds the left of the interaction; accountability (provenance) crowds the right; the middle — pre-interaction control over what may happen — is where today's stack is empty and where MAL sits. Because MAL's decisions are witnessed state transitions, occupying the middle yields the right-hand side as a corollary.

Three confusions, four commitments

The design commitments of Section 17 are the three confusions answered. Stateful authority (commitment 1) un-confuses the token from the relation. Instant revocation and the per-interaction witness (commitment 2) un-confuse session-grained access from interaction-grained control. Reconstructable evidence (commitment 4) keeps accountability a corollary of authority rather than a substitute for it. And the offline discipline (commitment 3) is the same separation, priced and disclosed, at the single point where it must be paid for.

19. Architectural principles

Principle 1 — Authority is a relationship, not a credential. A grant of authority is the establishment of a stateful relationship between a participant and an agent. It exists in the participant's context on MOI. It is not the same as the credentials, tokens, or signatures that reference it. Those are derived artifacts. The relationship is the truth.

Principle 2 — Separate the surface from the binding. What an agent could be authorized to do (the surface) is declared once, by the developer, at agent registration. What a specific participant authorizes a specific agent to do (the binding) is established per-participant, per-grant, at consent time. The surface is reusable; the binding is unique. Conflating them produces brittle authorization models — and the separation is exactly the two spines of Section 12: the surface lives on the agent's, the binding on the participant's.

Principle 3 — Deterministic enforcement, semantic interface. Runtime authorization decisions are deterministic and formally verifiable. The participant-facing surface — consent flows, intent questions, natural-language summaries — is semantic and LLM-mediated. The LLM helps the participant author intent. The LLM never decides whether an action is permitted.

Principle 4 — Witness, not bearer. Tokens issued by the authority layer prove that an action is valid against current state, at the moment of issuance. They are not transferable bearers of permission. They have no value detached from the state they witness. This is the structural defense against credential theft, replay, and accumulation.

Principle 5 — Disclosed offline. Offline action is a real requirement for some workflows. The system supports it, but only with explicit participant consent, narrow scope, short duration, and full disclosure of the revocation-blind window. The participant is told exactly what they are accepting. The system does not pretend offline tokens are revocable when they are not.

20. The three-component composition

MAL composes three roles, each filled by the component strongest at it: a decidable authority-surface language, MOI's stateful binding and witness, and an attenuable offline capability credential. The reference implementation binds each role to an existing component; the bindings are implementation decisions, recorded in the specification suite, and nothing in the architecture depends on them.

Role	What it does in MAL	Required properties	Why not used for the rest
Surface DSL	Authority schema. Declares the surface of what an agent may ever be authorized to do.	Decidable, formally verifiable, small grammar, a clean compile target for LLM-mediated consent, auditable.	Stateless by design. Cannot hold the participant's evolving authority state.
MOI	Stateful binding and per-interaction witness. Holds the participant's context, issues action witnesses, enforces revocation.	Participant-centric state, cryptographic evidence chain, signed state transitions, decentralized verification.	Not a policy language. Does not express the structure of authority surfaces.
Capability credential	Offline capability. Used only when the participant explicitly accepts offline action with disclosed risk.	Cryptographic attenuation, offline verifiability, capability-style semantics without a live decision point.	Bearer semantics. Used for the online path it would reintroduce the credential-vs-relationship confusion.

The composition rule

The surface DSL describes what authority can exist. MOI holds what authority does exist for each participant. The capability credential, only when explicitly enabled, carries a narrowly-scoped subset of that authority into offline contexts. Each role does exactly one job. None is asked to do what another does better.

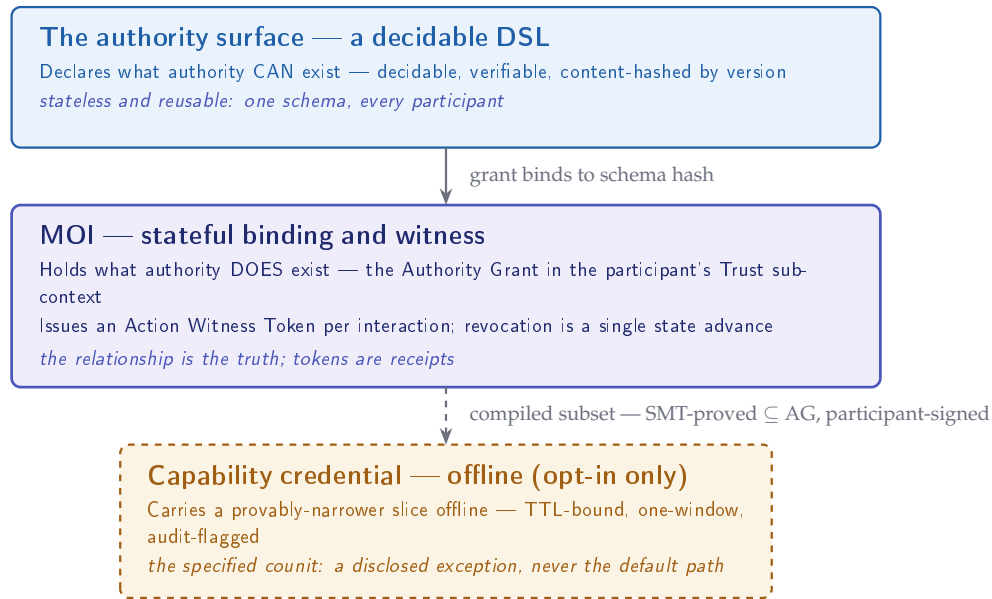


Figure 9. The three-role composition. The surface DSL declares what can exist; MOI binds it into participant state and witnesses every action; the capability credential — dashed because it is opt-in — carries a provably-narrower, participant-signed slice offline.

21. Vocabulary: mandates and their artifacts

The lite paper speaks in protocol vocabulary — mandates, witnesses, the Context Superstate. MAL speaks in artifact vocabulary — grants, witness tokens, credentials. They are the same system at two levels of formality, and the mapping is one-to-one. A *mandate* in the lite paper's sense ("agent: travel bot, scope: flights, limit: \$800, expires: Friday, revocable: immediate") is realized as an Authority Grant in the participant's Trust sub-context; the witnessed certification of each action is realized as an Action Witness Token issued by MOI through KRAMA's witness mechanism. The table below fixes the correspondence used in the rest of this paper.

Participant Layer (lite paper)	MAL artifact	Where it lives
Mandate (scoped authority)	Authority Grant (AG)	Participant's Trust sub-context, on the participant's spine
Agent's declared capability surface	Authority schema (content-hashed)	Agent's own context, on the agent's spine
Witnessed, certified action	Action Witness Token (AWT)	Issued per interaction by MOI; committed as a state transition in the participant's context
Bounded delegation chain	Delegated Action Token (DAT)	Minted by source agent; provably within the parent AG
(No online counterpart — offline only)	Offline Capability Credential (OCC)	Participant-signed capability credential; the licensed exception

22. Token taxonomy

The authority layer distinguishes four artifact types. Each has a different issuer, a different lifecycle, different revocation semantics, and a different audit weight.

Artifact	Issuer	Lifetime	Type	Revocation behavior
Authority Grant (AG)	Participant	Until participant revokes	Stateful record (not a token)	Revocation is structural — advances context version, invalidates all dependent artifacts.
Action Witness Token (AWT)	MOI	Single interaction	Ephemeral witness	Invalid immediately on AG revocation. Bound to context version at issuance.
Delegated Action Token (DAT)	Source agent	Single interaction	Subagent witness	Invalid on parent AG or parent AWT revocation. Provably a subset of the parent.
Offline Capability Credential (OCC)	Participant (signed)	Explicit TTL	Bearer-style capability credential	Revocation-blind within the TTL window. Window pre-accepted by the participant.

Three of these are tokens (AWT, DAT, OCC). The fourth — the AG — is a stateful record, the substrate from which all tokens derive. An AG cannot be lost or stolen because it is not transmitted. A token can be presented but is meaningless without resolution against the AG (for AWT and DAT) or without participant-signed scope (for OCC).

23. The authority lifecycle

Authority in MAL flows through five stages, from agent registration to action execution to revocation. Each stage has a defined trust event and a defined evidence artifact, and each evidence artifact is itself a witnessed state transition on the MDAG — the audit trail is the data structure, not a log beside it.

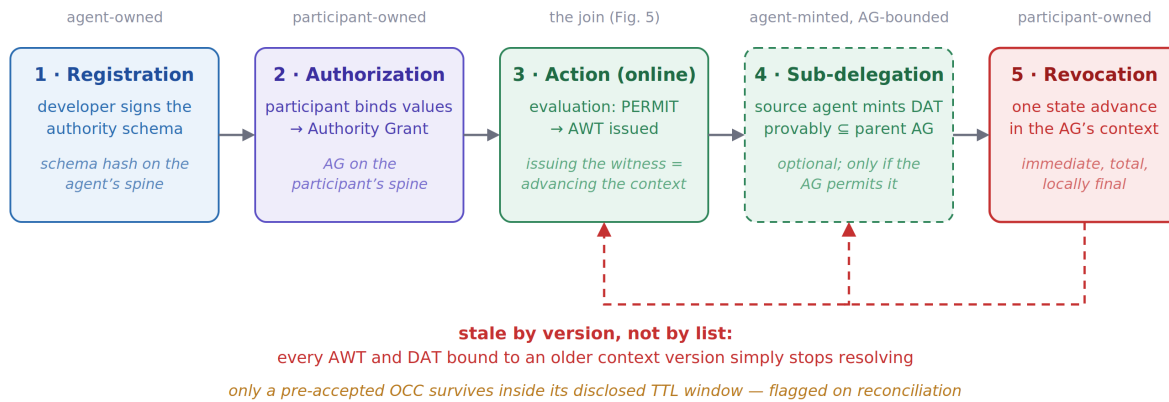


Figure 10. The authority lifecycle. Five stages, each a witnessed state transition on the spine that owns it. Revocation (stage 5) sweeps back by version: every AWT and DAT bound to an older context version stops resolving the moment a verifier consults MOI — no list, no TTL, no cache. Only a pre-accepted OCC survives inside its disclosed window, and is flagged on reconciliation.

Stage 1 — Agent registration

The agent developer declares an authority schema in the surface DSL, describing the agent's authority surface: the entity types it operates on, the actions it can perform, and the context attributes those actions are constrained by. The schema is versioned, signed by the developer's key, and registered immutably in the agent's own context. The schema also declares which attributes are participant-answerable at consent time, which are developer-fixed, and whether the agent supports offline capability (default: no) and sub-delegation (default: no).

Stage 2 — Participant authorization

A participant selects an agent. MAL walks the agent's schema and generates a consent flow — an LLM-mediated translation from the authority schema to natural-language intent questions. The participant answers. Their answers become bound values that, together with the schema reference, form an Authority Grant in the participant's context: schema reference, bound values, temporal bounds, sub-delegation flag, offline-capability flag, participant signature — stored as a state transition on the participant's spine. If the agent supports offline capability and the participant elects to enable it, a separate sub-flow handles OCC issuance (Section 24). If sub-delegation is supported and approved, the AG records this permission for use during action execution.

Stage 3 — Action execution (online)

The agent prepares an action: a target, a set of parameters. MAL retrieves the participant's AG, projects the relevant context state into the policy evaluation context, and runs the surface evaluation. If the action evaluates to PERMIT, MOI issues an Action Witness Token — a per-interaction artifact bound to the current context version — which the agent presents to the executing service or

counterparty. The AWT is a witness, not a permission. The act of issuing it is the same act as advancing the participant's context with the proposed action. The token is the receipt; the state transition is the authorization. This is Section 12's join, executed: agent schema hash on one side, participant grant on the other, validity computed where the spines meet.

Stage 4 — Sub-delegation

Many useful agentic workflows involve an autonomous agent invoking other agents — research subagents, payment subagents, data-fetching subagents — without each subagent being individually registered on MOI. Requiring full registration for every transient subagent would be operationally unworkable. Instead, when a registered source agent needs to invoke a subagent, and the participant's AG has authorized sub-delegation, the source agent mints a Delegated Action Token for the subagent. The DAT is provably a strict subset of the source agent's bound authority — narrower scope, optionally narrower time, never broader. Its name field identifies the subagent and its purpose, for evidence; its scope field is what authorizes the action. Surface evaluation against a DAT proceeds identically to evaluation against an AG, with the parent AG's bound values establishing the upper bound. The DAT becomes part of the parent action's evidence chain.

Stage 5 — Revocation

The participant revokes an AG by issuing a revocation transition on their context. The revocation is a single state advance; its effect is immediate and total. All future AWTs against the revoked AG are denied at issuance. All in-flight AWTs become invalid the moment they are presented to a verifier that consults MOI — their context version is now stale. All DATs derived from the revoked AG inherit the revocation transitively. All OCCs issued from the revoked AG remain technically signed but enter the revocation-blind window the participant pre-accepted.

Revocation is structural

There is no revocation list to propagate, no TTL to wait out, no cache to invalidate. The act of revoking is the act of advancing the context. The new context version is the authoritative truth; any token referencing an older version is, by definition, no longer current. This is hyper-local finality (Section 10) applied to authority: the revocation is final to its issuer the moment it is issued, because no third party's agreement was ever part of what made the grant true.

24. The offline discipline — a specified counit

Offline action is the only place in MAL where bearer-style semantics appear — and they appear only by explicit participant choice. This needs justifying, because the previous twenty pages argued that bearer semantics are exactly the copy map sneaking back in. The justification is the specified counit — the exception Part I licensed: a strictly linear category may admit a controlled exception on chosen objects, never a natural transformation on all of them. Burning a token is such an exception for value; the OCC is the corresponding exception for authority. It is not a natural bearer map quietly reintroduced into the category. It is a specified, participant-signed morphism on a

chosen grant: provably attenuated, time-boxed, one-window, and audit-flagged. The discipline below is the operational form of that categorical restriction — the exception is real, narrow, and visible, which is exactly what distinguishes a specified counit from a structural one.

When OCCs are issued

Offline Capability Credentials are issued only when three conditions are met: the agent's authority schema declares offline capability as supported; the participant explicitly opts in during AG authorization; and the participant signs the resulting OCC, having seen its scope, duration, and the revocation-blind window they are accepting. Defaults are off at every layer. Participants who never enable offline capability never have OCCs in circulation.

OCC properties

- **Provably narrower than the source AG.** The OCC compilation pipeline runs an SMT-backed subset proof: every action permitted by the OCC is permitted by the AG at issuance, with at least one constraint strictly narrower. If the proof fails, the OCC is not issued.
- **Participant-signed.** Only the participant can sign an OCC. MAL prepares the credential; the participant signs it. This is the inviolable line.
- **Time-bound and one-window.** OCCs have explicit TTLs. Within the TTL, the OCC is valid for the actions in its scope. After the TTL, it is invalid for everything.
- **Reconciled on reconnect.** Actions executed under an OCC are reported back to MOI when the agent reconnects, and committed to the participant's evidence log.

The revocation-blind window

If the participant revokes the source AG during the OCC's TTL, in-flight OCC actions can still be honored by offline verifiers until either the TTL expires or the agent reconnects and learns of the revocation. This is the unavoidable cost of offline action. MAL does not hide this cost. It surfaces it during consent: the participant is told, in plain language, that revocation during the OCC window cannot stop actions within the window's scope. They accept this risk explicitly, with the scope and duration they chose, or they decline to enable offline capability. On reconciliation, any actions executed after the revocation are committed to the evidence log with an explicit flag — `executed_post_revocation`. The participant sees them in their audit trail with that flag. The record is complete either way.

The offline trade is surfaced, not hidden

Every authorization system has a revocation-blind window; most do not tell the user about it. In MAL the window is participant-chosen, scope-bounded, and audit-flagged. The cryptography for offline action is unchanged; the discipline around it is what changes.

25. Evidence and audit

Every artifact in the authority layer generates evidence; the evidence log on MOI is the structural consequence of how the layer works. Because every authority decision is a witnessed state transition on the MDAG, the audit trail and the state history are the same object read two ways.

For every action taken by an agent on behalf of a participant, the evidence log reconstructs: which Authority Grant authorized the action; which authority schema and bound values were in force at the moment of authorization; which context state was projected into the policy evaluation; which artifact type (AWT, DAT, OCC) carried the authorization; which counterparty or service acknowledged the action; and whether the action was executed online, offline, or under sub-delegation — together with any applicable flags (post-revocation, reconciled). Most authorization audit logs answer what happened. The MAL evidence log answers why it was permitted at the moment it happened, in cryptographically verifiable form.

A claim the layer makes possible

The set of actions an agent has performed under a participant's authority is, at any moment, exactly the set of actions the participant would currently approve given everything they know — except for actions taken in offline windows the participant explicitly chose to accept, which are themselves clearly marked in the audit trail.

26. MAL in action: one mandate, two worlds

The mandate this paper has carried since Section 12 now runs end to end, twice: once on today's authorization stack, once on MAL. The walkthroughs are step-for-step mirrors; each step in the first is annotated with the confusion it executes.

The use case

Asha is the participant. Her mandate: “Book me the cheapest Delhi–London flight, under \$800, pay with my Visa ending 6411, before Friday.” **TravelBot** is the registered agent acting for her — flight search, booking, and payment up to a participant-set bound; it supports sub-delegation; offline capability is not enabled. **FareScan** is a transient search subagent TravelBot invokes for fare discovery. **PayFlow** is the payment subagent that executes the charge. The **airline** is the counterparty that must decide whether to honor the booking. The week: the mandate is given on Monday; on Tuesday Asha lowers her budget to \$500; on Thursday a \$480 fare is booked. A branch point on Wednesday — Asha cancels the trip entirely — is played in both worlds.

How it works today

1. **Monday — consent.** Asha connects TravelBot through an OAuth flow. The scopes on offer are the provider's, not hers: “manage bookings,” “process payments.” There is nowhere to say \$800, nowhere to say Friday, nowhere to say which card. She over-grants on day one, because the grain of consent is the scope, not the intent. [Confusion 2: the consent event is session-grained]

access control standing in for interaction-grained authority.]

2. **Token issuance.** TravelBot receives access and refresh tokens — standing authority, cached in the agent’s infrastructure. From this moment Asha’s actual intent exists in two places: her head, and nowhere enforceable. *[Confusion 1: the token is a snapshot of Monday, and nothing binds it to what she wants now.]*
3. **Search fan-out.** TravelBot hands FareScan a copied API key so it can query fare systems. Asha cannot see this hop; nothing required her to be asked. The key is information, and information copies freely. *[Confusion 1, executing as failure 4: opaque sub-delegation.]*
4. **Tuesday — the budget change.** Asha decides \$500 is the limit. There is no enforceable place for the number to live. She can tell the bot — a prompt, not a control. She can edit a setting in TravelBot’s database — the application’s policy applied to her, not hers applied to it. The tokens, which are what verifiers actually check, are unchanged. *[Confusions 1 and 2 together: her intent has no address, and no channel runs at the speed of the agent.]*
5. **Thursday — the booking.** PayFlow charges the card under the payments scope. The airline’s systems see a valid token and honor it. They cannot ask “is this within what she wants today?” — there is no bound to check, because the bound was never representable. Had FareScan surfaced a \$799 fare instead of \$480, it would have been booked: the token still says yes. *[Confusion 1: the verifier is reading information about Monday.]*
6. **Wednesday branch — revocation.** Asha cancels the trip. She must find the revocation screen for TravelBot, trust TravelBot to un-grant FareScan and PayFlow, and wait out token lifetimes; PayFlow’s cached token survives until its TTL expires. Her revocation propagates at human speed against agents acting at machine speed — the window between the two is exactly where the wrong booking happens. *[Confusion 2, executing as failures 1 and 2.]*
7. **Friday — the audit.** What did the agents do this week, and on what basis? The answer is scattered across TravelBot’s logs, FareScan’s logs, PayFlow’s processor records, and the airline’s systems — each recording what it chose to, none recording why an action was permitted, nothing binding them together. If a charge looks wrong, reconstruction is forensic archaeology. *[Confusion 3: identity exists — she knows who acted — and partial records exist; control never did, and the records cannot substitute for it.]*
8. **Net position:** recognition without control, records without prevention, intent without an address.

How it works with MAL

1. **Registration (before Monday).** TravelBot’s developer declares an authority schema: flight search, booking, payment up to a participant-bound amount; sub-delegation supported; offline off. The schema is signed, content-hashed, and registered on TravelBot’s own spine. The surface of what could ever be authorized now exists — once, for every participant.

2. **Monday — consent.** MAL walks the schema into intent questions, and Asha answers in her own terms: under \$800, Visa ending 6411, before Friday. Her answers become bound values; the Authority Grant is committed to her spine. Her intent now has an address — authored once, at human speed, in her context and no one else's.
3. **Search fan-out.** TravelBot mints a Delegated Action Token for FareScan: search only, no payment authority, provably a subset of the grant. The hop is on the evidence chain by construction. Nothing copyable confers power; what FareScan holds is meaningless except against Asha's current state.
4. **Tuesday — the budget change.** One state advance in Asha's context: the bound value moves from \$800 to \$500. Nothing propagates, because nothing needs to — from this instant, every evaluation anywhere witnesses against \$500. The change is enforced at machine speed because enforcement was never waiting on a human to begin with.
5. **Thursday — the booking.** FareScan surfaces a \$480 fare. The surface evaluation runs against the current grant: PERMIT. MOI issues an Action Witness Token bound to the current context version; PayFlow executes under a payment DAT provably inside \$500 and Visa 6411. The airline verifies a witness of current state — not a memory of Monday's consent. A \$799 fare on Wednesday would have been denied at issuance: nothing to roll back, because nothing happened.
6. **Wednesday branch — revocation.** One revocation transition on Asha's context. Future witnesses are denied at issuance; any in-flight token is stale by version the moment it is presented; FareScan's and PayFlow's DATs inherit the revocation transitively. Nothing to chase, no TTL to wait out, no screen-hunting — final to Asha the instant she issues it, because no third party's agreement was ever part of what made the grant true.
7. **Friday — the audit.** Asha's evidence log reconstructs the week: which grant authorized each action, which schema version and bound values were in force, which DAT carried which subagent's hop, which counterparty acknowledged. Not just what happened — why it was permitted at the moment it happened. Accountability arrived as a corollary; no one built it separately.
8. **Net position:** intent with an address, control before the act, evidence after it — the three concerns of Section 18 separated, each in its place.

Moment	Today	With MAL	Confusion resolved
Consent	coarse provider scopes; over-grant on day one	schema → intent questions → bound values in the AG	2 — access vs. authority
Where intent lives	nowhere enforceable; tokens are snapshots	Authority Grant on the participant's spine	1 — information vs. authority
Subagent hop	copied key, invisible to the principal	DAT provably $\subseteq$ AG, on the evidence chain	1 — plus visible delegation
Mid-week change	unenforceable; verifiers read stale tokens	one state advance; instant everywhere	1 + 2 — machine-speed enforcement
The booking	token says yes regardless of current intent	witness of current state at issuance	1 — witness, not bearer
Revocation	screens, TTLs, surviving cached tokens	stale by version at next presentation	2 — no human-speed window
The audit	scattered whats, no why, nothing binding	reconstructable why-permitted, one log	3 — accountability as corollary

The week is deliberately ordinary — that is the point. No exotic adversary appears, no protocol is attacked; a person simply changes her mind twice, the way people do. On today's stack, each change of mind opens a window in which agents act on information about an intent that no longer exists. On MAL, there is no window, because there was never a copy: the agents were always acting against the relation itself. Every difference between the two columns above is the same difference — where authority lives, and at what speed it is enforced.

The same week, multi-agent

The walkthrough above is one agent; the orchestration topology of Section 16 runs identically, and closes that section's open window. Priya's grant — procurement up to \$52,000, cost center CC-4417, by the 30th — is joined by ProcureBot, which spawns three negotiator subagents at runtime, none of them registered, each holding a DAT provably carved from the live grant. Her mid-plan tightening, \$52,000 to \$45,000, is one state advance: from that instant every evaluation in the tree — including agents spawned afterward — witnesses against the new head, with nothing reissued because nothing was ever copied. Revoking one negotiator's lineage kills that branch transitively and leaves its siblings untouched. The plan completes inside the bound, with a per-action evidence chain answering why each step was permitted at the moment it ran. Figure 11 shows the run: execution is a stream-stream join against the live head of the authority stream — no table of issued tokens, no TTLs, no reissuance — the problem topology of Figure 7 with the window deleted.

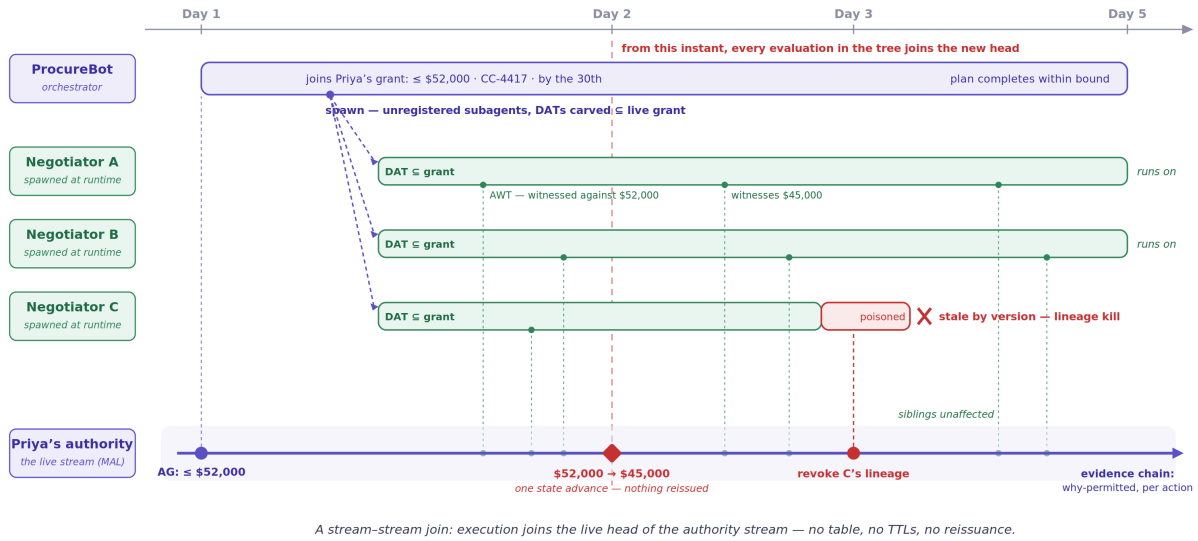


Figure 11. The orchestration topology on MAL. ProcureBot joins Priya's live grant; runtime-spawned negotiators hold DATs provably within it; her \$52,000 $\rightarrow$ \$45,000 tightening is one state advance that every evaluation in the tree joins from that instant; revoking one lineage poisons that branch by version and leaves siblings unaffected. A stream-stream join: no table, no TTLs, no reissuance.

Part VI — Against Today's Systems

27. Four generations of agent authority

Part V specified the architecture; this Part reads the industry against it. The reading is generational rather than vendor-by-vendor, because agent authorization has been rebuilt four times, and each generation genuinely fixed its predecessor — while leaving one thing unchanged.

The key — bearer tokens: API keys, JWTs, OAuth-style credentials. What the verifier checks is *the paper*: possession. The generation fixed portability — a key works anywhere, even offline. What still breaks: the key is detached from the principal by design, so a copy is as good as the original and both are stale from the moment they are minted; any agent holding the key holds all of her authority, including helpers she has never heard of.

The doorman — identity and access: the IdP era. What the verifier checks is *the face*: is this agent on the list. The generation fixed impostors — real names, real bans, strike anyone off the roster. What still breaks: the doorman guards the door, not the deed. The list has no column for “\$800”; it was never about her intent. Only faces on the list exist, so a helper hired mid-task either borrows the agent's badge or stays out.

The guard with the rulebook — runtime policy: the PDP era. What the verifier checks is *the rules*: a fresh decision on every request, nothing acting on stale information at decision time. The generation fixed the staleness of the *decision*. What still breaks is the authorship of the *book*: the organization's rules about the organization's agents, written by administrators, about kinds. It cannot hold her pen, her terms, or her counterparties — it governs their agents; it was never her authority over any agent. An agent born this morning is not in the book; it can act only as a kind, inheriting role-shaped power. Section 18's second confusion, institutionalized.

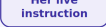
The power of attorney — the participant's live grant: MAL. What the verifier checks is *her live instruction*: authored by her, in her own terms, indexed to her, evaluable toward any agent — including one that did not exist this morning, which receives a slice provably inside her bound and never a copy of her power. The author has a home; keeping her instruction true is her job, exactly where the job belongs.

The Tuesday test

One event separates the four generations: at 9:14 on a Tuesday, the principal changes \$800 to \$500. Generation one — the key still says \$800, to every door, until it expires. Generation two — nothing to update; her number was never on the list. Generation three — she files a ticket; the book changes when an administrator edits it, eventually. Generation four — one edit; every check, everywhere, including agents spawned after the edit, sees \$500; nothing reissued, because nothing was ever copied.

What does the system actually check?

Four generations of agent authority — and the one test that tells them apart: Tuesday, 9:14 am, she changes \$800 → \$500.

The method	What it checks	What it fixed	What still breaks	Tuesday, 9:14 am — the test
 The key bearer tokens — JWT, API keys	 The paper	Portability — works anywhere, even offline	Detached from her by design: a copy is as good as the original — stale from the moment it's minted. <i>Any agent holding the key holds all of her authority, including helpers she's never heard of.</i>	 The key still says \$800 — to every door, until it expires
 The doorman identity & access — the IDP era	 The face	No impostors. Real names, real bans — strike anyone off the list	Guards the door, not the deed — and the list has no column for "\$800"; it was never about her intent. <i>Only faces on the list exist: a helper hired mid-task borrows the agent's badge — or stays out.</i>	 Nothing to update. Her number was never on the list
 The guard with the rulebook runtime policy — the PDP era	 The rules	A fresh decision, every request — nothing acts on stale info at decision time	The org's rules about the org's agents — written by admins, about kinds. It cannot hold her pen, her terms, or her counterparties. It governs their agents; it was never her authority over any agent. <i>An agent born this morning isn't in the book — it can act only as a kind, inheriting role-shaped power.</i>	 She files a ticket. The book changes when an admin edits it — eventually
 The power of attorney the participant's live grant — MAL	 Her live instruction	The author has a home. Her intent, in her own terms, indexed to her — toward any agent	Only what should remain: keeping her instruction true is her job — exactly where the job belongs. <i>An agent that didn't exist this morning gets a slice provably inside her bound — never a copy of her power.</i>	 One edit. Every check, everywhere — including agents spawned after — sees \$500. Nothing reissued; nothing was ever copied

Each generation fixed the one before it. None changed what was being checked — paper, face, rules: content, authored by someone else, indexed to someone else's roster, stale against her.

Every generation asked content to do a person's job — until the content got an author.

Figure 12. Four generations of agent authority and the one test that tells them apart. Each generation fixed the one before it; none changed what is being checked — paper, face, rules are all content, authored by someone other than the principal and stale against her. The fourth generation changes the category: the check lands on her live instruction — the relation itself.

The pattern the ladder exposes is the one this paper opened with. Paper, face, rules: all three are *content* — authored by someone other than the principal, indexed to someone else's roster, Cartesian and therefore stale against her the moment she changes her mind. Every generation asked content to do a person's job. The fourth is not a better artifact; it is the first change of category. The check lands on the relation itself — her live instruction, the linear thing — and every artifact in the system is demoted to a witness of it.

None of this displaces the identity layer. Recognition remains necessary, and the enterprise identity stack — directories, single sign-on, session security, lifecycle management — keeps its public contract unchanged. MAL sits beneath it as the relation: the identity layer authenticates *who is acting*; the authority layer holds *what the principal's live instruction permits*. The two compose. The ladder's fourth rung is an addition to the stack, not a replacement of its third.

28. Properties compared

Read as rungs of the ladder, the alternatives sort by which generation they perfect — and the trilemma explains why perfecting a rung never reaches the fourth. The table below compares MAL's core properties to the dominant alternatives currently in use or under active development.

The pattern is the trilemma of Section 14 made empirical: each alternative holds some of the properties and structurally cannot hold all of them, because each stores or bears the relation rather than composing it from participant-indexed state.

Property	OAuth / JWT	UCAN	Biscuit alone	OPA / Rego	MAL
Authority lives in state, not token	No	No	No	Partial	Yes
Instant revocation (no TTL wait)	No	No	No	Depends	Yes
Sub-delegation provably bounded	No	Yes	Yes	Hard to express	Yes
Decidable policy evaluation	N/A	N/A	Yes (Datalog)	No	Yes
Disclosed offline model	No	No (implicit)	Yes	N/A	Yes (opt-in)
Complete why-permitted audit	No	Partial	Partial	Partial	Yes

Part VII — Conclusion and the Series

29. Conclusion

The architecture described in this paper is not a set of independent design decisions that happen to compose well. It is a sequence of corollaries of a single categorical fact. Information is Cartesian and value is linear; a linear resource stored in a Cartesian substrate must have its copy map forbidden after the fact, and global consensus is that patch. Index state by participant instead, and value becomes a genuine linear resource with no copy map to forbid — so reentrancy, aliasing, double-spend, and the need for global ordering do not arise. The MDAG is the string diagram of that linear category; the Interaction Context Space is its tensor structure; the no-reach-back invariant is the absence of the copy map enforced operationally.

Authority is where the argument stops being abstract. A grant of authority is a many-to-many relation with two independent writers — the one structure an ownership model cannot hold without surrendering parallelism, agent-side consistency, or clean revocation. On the participant-indexed substrate it is not held at all: it is composed, at the moment of action, from two contexts each side owns outright. The MOI Authority Layer is that composition engineered — a decidable surface describing what authority can exist, MOI holding what authority does exist, a participant-signed capability credential carrying a narrow slice of it offline under a discipline the theory itself licenses. Tokens witness; state authorizes; revocation is a local fact; the audit trail is the data structure. Information about authority is not authority — and on this substrate, for the first time, the two can be different things. Each of the four design commitments of Section 17 is a structural property of Parts I through IV wearing operational clothes.

And the proof at the top of the ladder is a proof of the whole ladder. Part II ordered the classes of value by writer structure — owned, attested, contextual, relational — and the substrate has now been shown to hold the maximal class intact. Everything below follows by containment: native assets are the one-writer case the MDAG already carries on every spine; verifiable credentials are the issuer–holder relation with the attestation kept live; participant memory is contextual value resident where interactions land. The papers that develop those classes — participant-indexed exchange and native assets, verifiable credentials (the Proofly series), participant memory and agent context — are extensions of this one: the same classification, instantiated down the ladder on the same substrate.

The claim is therefore not that MOI is a faster machine of the familiar kind, but that it is a different substrate: one whose state space is participant-indexed and whose computational space is constituted per interaction. Parallelism, no fragmentation, hyper-local finality, modulated trust, and structural authority are not five features to be defended individually. They are five shadows of one decision — to treat WHO as a dimension of computation rather than a label within it — and that decision is, given the linearity of value, not a preference but a requirement.

Information can be processed anonymously. Value requires participants. The lite paper stated it; this paper has tried to show why it is a claim about substrates rather than a slogan about markets. The

research series that follows proves it.

The series

This paper is one of a series. The formal development of the theory and the protocol specifics are in the Contextual Compute research papers (CC0–CC6).

Sarva Labs • Princeton, NJ • August 2026 • moi.technology